A Post-Processing Fairness Mitigation Method for Medical Prediction Models - Supplementary Materials

Noga Stern, Inbal Livni Navon, Omer Reingold, Eitan Bachmat, Noam Barda* and Noa Dagan*

Appendix I
Fairness Definitions

All the following fairness definitions can be expressed with the following notations:

X - patients' data, Y - labels, $\hat{Y}$ - predicted labels (0 or 1), A - protected attributes (such as gender or race).

Using these notations, some of the most common fairness definitions are:

Demographic Parity - The proportion of those predicted positive must be equal across all groups [1]. Formally, $\forall a, a' \in A : Pr(\hat{Y} = 1|A = a) = Pr(\hat{Y} = 1|A = a')$

Equal Opportunity - The sensitivity (recall) must be equal across all groups [2]. Formally, $\forall a, a' \in A : Pr(\hat{Y} = 1|Y = 1, A = a) = Pr(\hat{Y} = 1|Y = 1, A = a')$

Equalized Odds - The specificity and sensitivity must be equal across groups [2]. Formally, $\forall a, a' \in A : Pr(\hat{Y} = 1|Y = y, A = a) = Pr(\hat{Y} = 1|Y = y, A = a')$

Appendix II
Branch and Bound Algorithm

In this method, we aim to find the optimal solution with respect to equal opportunity. Since there is a finite number of ways to select patients for the intervention group, there are also a finite number of possible solutions. This makes it theoretically possible to find the optimal solution using brute force. A naive approach would involve evaluating all possible combinations of patients in the intervention group and selecting the combination that achieves the highest fairness. However, this approach is computationally inefficient, particularly for large datasets. As a result, a more efficient method is required.

Noga Stern, Inbal Livni Navon and Eitan Bachmat are with the Stein Faculty of Computer and Information Science, Ben-Gurion University of the Negev, Be'er Sheva 84105, Israel.

Omer Reingold is with Stanford University, Stanford, CA 94305 USA

Omer Reingold is with Stanford University, Stanford, CA, USA (email: omreing@stanford.edu).

Noam Barda is with the Stein Faculty of Computer and Information Science and the department of Epidemiology, Biostatistics and Community Health Services, Ben-Gurion University of the Negev, Be'er Sheva 84105, Israel, and also with Infection Control & Prevention Unit, Sheba Medical Center, Ramat Gan 52621, Israel.

Noa Dagan is with the Stein Faculty of Computer and Information Science, Ben-Gurion University of the Negev, Be'er Sheva 84105, Israel, and also with Clalit Research Institute, Innovation Division, Clalit Health Services, Tel Aviv 62098, Israel

*These authors contributed equally.

Branch and Bound is a technique for solving optimization problems by dividing them into smaller sub-problems and using a bounding function to discard sub-problems that cannot contain the optimal solution [3].

Our implementation leverages an important observation: the number of possible sensitivity combinations is typically much smaller than the number of possible patient combinations. For instance, there may be a range of patient counts for a group (e.g., 10 to 15) that yields the same sensitivity, provided that patients 11–15 did not die. This implies that the number of unique sensitivity values for a group is smaller than the total number of patients in that group.

Consequently, it is more efficient to focus on combinations of group sensitivities rather than individual patient combinations. By reducing the number of combinations that need to be evaluated, this approach significantly improves computational efficiency.

The algorithm consists of two main parts:

1) **Finding a Candidate Sensitivity Combination:** Identify a candidate for the optimal sensitivity combination.
2) **Validating the Candidate:** Take the candidate as input and verify whether there is a valid combination of patients that produces this sensitivity combination. A valid combination is one that includes exactly the required number of patients to fill the intervention group.

A. Part 1: Finding a candidate sensitivity combination

The process for identifying a candidate sensitivity combination is as follows:

1) **Initialize:** Run the iterations-based algorithm for a predefined number of iterations and store the current smallest difference between the maximum and minimum sensitivity values as curr_min.
2) **Record Sensitivity Values:** For each group, evaluate all possible ways to select patients for the intervention. Consider every possible selection, from choosing no patients to choosing the maximum number of patients from that group, and record the corresponding sensitivity value for each choice.
3) **Explore Sensitivity Combinations:** Iterate over all possible combinations of sensitivity scores across groups. For each combination, calculate the difference between the highest and lowest sensitivity scores. If this difference is smaller than curr_min, check whether there

is a valid combination of patients that achieves this sensitivity combination. If such a valid combination is found, update curr_min and store the combination as the current candidate.

It is not necessary to examine all possible sensitivity combinations due to the curr_min constraint. For instance, consider a scenario with 5 groups, each having 10 possible sensitivity scores, and a curr_min value of 0.4. If group A has a sensitivity score of 0.2 and group B has a score of 0.8, the difference between these scores (0.6) already exceeds curr_min. Thus, no combination that includes these scores can be optimal, eliminating the need to evaluate them further.

4) **Determine Group Thresholds:** Once the optimal sensitivity combination is identified, determine the corresponding patient combination. For each group, if n patients are included in the combination, set the group threshold to just below the predicted probability of the n-th patient.

B. Part 2: Validating a Sensitivity Combination

To confirm whether a sensitivity combination is valid:

1) **Identify Patient Ranges:** From step 2 in Part 1, determine the ranges of patient counts that could produce the given sensitivity scores for each group.
2) **Evaluate Combinations:** Iterate over all possible combinations of patient counts within the identified ranges. For each combination, calculate the total number of patients included.
3) **Check Validity:** A sensitivity combination is valid if and only if the total number of patients in the combination equals the intervention group size.

Appendix III
Buckets Algorithm

The Buckets Algorithm (see Algorithm **??** for pseudocode) begins by determining the sensitivity range as follows:

1) **Constructing Buckets:** Define several "buckets," where each bucket corresponds to a sensitivity score range. The size of this range is called the "bucket width." Each bucket represents a proposed solution, answering the question: for each group, what is the maximum number of patients that can be included in the intervention group such that the group's sensitivity score remains within the bucket's range? For example, suppose the 0.15–0.16 bucket contains the number 50 for male patients and the number 35 for female patients. This indicates that including 50 male patients and 35 female patients in the intervention results in a sensitivity score between 0.15 and 0.16 for both groups. Adding even one more patient—51 males or 36 females—would result in a sensitivity score exceeding 0.16 for the respective group.
2) **Binary Search for Sensitivity Score:** Use binary search to identify a sensitivity score that achieves the exact intervention group size, meaning the number of patients predicted to be "high-risk" matches the target size. The following iterative process is used: **Step 1**: Select an initial sensitivity score (sensitivity_score), starting with 0.50. **Step 2**: Identify the bucket whose range includes sensitivity_score. For instance, bucket 50 might represent the range 0.50–0.51, which contains 0.50. **Step 3**: Calculate the total number of patients from all groups within this bucket. For example, if the male group contributes 100 patients and the female group contributes 90, the total is 190. **Step 4**: Check the following conditions:
   a) If the total number of patients equals the intervention group size, the correct bucket has been identified, and the process terminates.
   b) If the total number of patients is smaller than the intervention group size, and the bucket above this one is in the "too-large" list, the process also terminates.
   c) If the total is too small but the next bucket is not in the "too-large" list, add the current bucket to a list of "too-small" buckets, increase sensitivity_score, and repeat from Step 1.
   d) If the total is too large, and the bucket below this one is in the "too-small" list, the process terminates.
   e) If the total is too large but the bucket below it is not in the "too-small" list, add the current bucket to a list of "too-large" buckets, decrease sensitivity_score, and repeat from step 1.

   Throughout this loop, two bounds are maintained: the upper bound, initialized to 1, and the lower bound, initialized to 0. During each iteration, these bounds are updated: - If the bucket is too small, set the lower bound to the current sensitivity_score. - If the bucket is too large, set the upper bound to the current sensitivity_score. After updating the bounds, calculate the new sensitivity_score as the average of the upper and lower bounds.
3) **Final bucket determination:** Once the appropriate bucket is identified, there are two possible scenarios:
   - **Exact Fit:** If the bucket matches the exact target size, the algorithm terminates, and the patients within this bucket are selected for the intervention.
   - **Adjustment Required:** If the bucket does not perfectly match the target size, the algorithm identified one bucket that is too small and another, right above it, that is too large. All patients from the smaller bucket are included in the intervention, and additional patients are selected from the larger bucket using a branch-and-bound algorithm (described in Appendix II).
4) **Setting group cutoffs:** Finally, the solution specifies the number of patients from each group to include in the intervention. This is translated into cutoff values for each group, with the cutoff set just below the lowest predicted probability of a patient from that group who is included in the intervention.

This solution is not guaranteed to be optimal; however, the error is bounded by at most $2 \cdot bucket_width$ (recall that the buckets are defined by a range of possible sensitivity values).

To demonstrate this, we consider the two conditions under which the bucket search stage of the algorithm terminates:

1) **Exact Match:** The algorithm identifies a bucket containing the exact number of patients needed to fill the intervention group. In this case, the solution is entirely within one bucket, so the difference between the highest and lowest sensitivities is at most $bucket_width$.
2) **Split Between Buckets:** The algorithm identifies a bucket containing fewer patients than required and supplements it with patients from the next higher bucket. Here, the solution spans two buckets, meaning the difference between the highest and lowest sensitivities is at most $bucket_width$.

Thus, the error is also bounded by $2 \cdot bucket_width$. It is important to note that this proof assumes the ability to achieve each sensitivity range for all groups. However, this assumption may not always hold. For example, consider a bucket width of 0.1. If adding X male patients to the intervention results in a sensitivity of 0.45, but adding X+1 male patients results in a sensitivity of 0.65, then the 0.5–0.6 bucket will contain no male patients.

Our implementation addresses this issue as follows: if the bucket search stage identifies a bucket that has an "empty" group (e.g., the 0.5–0.6 bucket in the example above), the algorithm doubles the bucket width and reruns the search.

While theoretically, this process could require repeatedly doubling the bucket size, potentially reducing efficiency, in practice, this issue is rare. In our experiments, we initialized the bucket width to 0.01, and in most cases, no adjustments were necessary.

Appendix IV
Iterative Algorithm

This implementation iteratively adjusts the composition of the intervention group to achieve a fairer result. Specifically, in each iteration, a patient from the group with the highest sensitivity is removed from the intervention group, and a patient from the group with the lowest sensitivity is added to it. The pseudocode for this algorithm is presented as Algorithm 1. It is worth noting that this "greedy" algorithm does not guarantee an optimal solution.

The implementation begins with the initial results from the prediction algorithm, where a single cutoff is applied across all groups. A patient is included in the intervention group if and only if their predicted probability exceeds this threshold. During each iteration, the algorithm performs the following steps:

1) **Evaluate Sensitivity:** Compute the sensitivity value for each group.
2) **Add a Patient from the Lowest Sensitivity Group:** Identify a patient from the group with the lowest sensitivity who is not currently in the intervention group and add them to it. Specifically, select the patient closest to the threshold who is still below it, as this individual is considered by the model to be at the highest risk among those not currently included.

Algorithm 1 Iterative post-processing algorithm

Input A list of groups G and a set D=$\{y_i, \hat{p}_i, \hat{y}_i, g_i\}$, where $y_i \in \{0,1\}$ is a label, $\hat{p}_i \in [0,1]$ is a predicted probability, $\hat{y}_i \in \{0,1\}$ is a predicted label and $g_i \in G$ is a group

Output $\hat{Y}$ Updated predicted labels

sensitivities $\leftarrow \{\}$
for $t \leftarrow 1$ to T **do**
  for $g \in G$ **do**
    $sensitivities[g] \leftarrow \frac{|\{i|g_i=g \wedge \hat{p}_i=1 \wedge y_i=1\}|}{|\{i|g_i=g \wedge y_i=1\}|}$ ▷ step 1
  end for
  $group_{min} \leftarrow argmin(sensitivities)$ ▷ step 2
  $j = \{argmax(\hat{p}_i) \mid g_i = group_{min} \wedge \hat{y}_i = 0\}$
  $\hat{y}_j \leftarrow 1$
  $group_{max} \leftarrow argmax(sensitivities)$ ▷ step 3
  $k = \{argmax(\hat{p}_i) \mid g_i = group_{max} \wedge \hat{y}_i = 1\}$
  $\hat{y}_k \leftarrow 0$
end for

Due to the definition of sensitivity (the ratio of patients in the intervention group who died to the total number of patients who died in the group), this step can only increase or maintain the sensitivity. If the selected patient died, the sensitivity increases; if not, it remains unchanged. Crucially, this step never decreases sensitivity. In other words, this adjustment either moves closer to satisfying equal opportunity or leaves the fairness unchanged, but it does not make it worse.

3) **Remove a Patient from the Highest Sensitivity Group:** Identify a patient from the group with the highest sensitivity who is currently in the intervention group and remove them. Specifically, select the patient closest to the threshold, as they are considered by the model to be at the lowest risk among those included in the intervention group.
This adjustment either decreases the sensitivity for the group or does not change it. While decreasing sensitivity is generally undesirable, this step is necessary to maintain the intervention group size.
4) **Update the Solution:** After these adjustments, the algorithm produces a new solution with the same number of patients in the intervention group. The new solution is typically at least as fair as the previous one and may be fairer. This process is repeated for a predetermined number of iterations.

At the conclusion of the algorithm, the solution specifies the number of patients from each group to be included in the intervention. This can be converted into group-specific cutoffs: for each group, set the cutoff to just below the lowest predicted probability of a patient from that group who is included in the intervention.

Appendix V
Study Population

Table I shows the study population, focusing on the intersection between protected groups.

TABLE I: Study population - intersection groups

		MIMIC			GOSSIS		
Variable	Level	Total	Train	Test	Total	Train	Test
Age group - Race (%)	Under 60 - Black/African American	3.67	3.48	3.86	-	-	-
	Under 60 - Other	6.88	6.87	6.89	-	-	
	Under 60 - White	19.51	19.53	19.49	-	-	-
	60-75 - Other	5.92	5.91	5.93	-	-	-
	60-75 - White	25.17	25.1	25.23	-	-	-
	Over 75 - Other	4.64	4.71	4.57	-	-	-
	Over 75 - White	20.05	20.09	20.01	-	-	-
	Other	14.17	14.32	14.02	-	-	-
Age group - Apache 3j body system (%)	Under 60 - Cardiovascular	-	-	-	9.95	10.12	9.78
	Under 60 - Metabolic	-	-	-	6.58	6.65	6.51
	Under 60 - neurological	-	-	-	5.34	5.38	5.29
	Under 60 - other/unknown	-	-	-	4.78	4.87	4.68
	Under 60 - respiratory	-	-	-	4.61	4.57	4.66
	60-75 - Cardiovascular	-	-	-	14.28	13.96	14.6
	60-75 - Respiratory	-	-	-	5.06	4.98	5.13
	60-75 - Sepsis	-	-	-	4.8	4.78	4.82
	Over 75 - Cardiovascular	-	-	-	8.87	8.77	8.96
	Other/Unknown	-	-	-	35.75	35.92	35.58

Appendix VI
Logistic Regression Results

A logistic regression model with L1 regularization was applied to both the MIMIC and GOSSIS datasets. Tables II and III present the estimated coefficients.

Appendix VII
Post-processing Algorithms Runtime

While in with the MIMIC dataset all three algorithms required similar execution times, when optimizing over the GOSSIS dataset the branch-and-bound algorithm took over 24 hours to complete. In contrast, the iterative algorithm and the buckets algorithm completed running in approximately 35 seconds and 2 minutes, respectively. Table IV shows runtime comparison.

Appendix VIII
Feature Importance

Fig. 1 shows feature importance for the base model, as ranked by SHAP.

Appendix IX
ROC

Fig. 2 shows ROC curves for the base model.

Appendix X
Calibration

Fig. 3 shows calibration curves for the base model.

Appendix XI
Train-test split sensitivity analysis – fairness results

Figure 4 shows the fairness results obtained using an 80:20 train-test split and a logistic regression model. The results show an average decrease of 48% in the span of sensitivity values across sub-groups, compared to an average decrease of 56% with the 50:50 train-test split. Although the reduction is smaller with this split, it remains substantial and demonstrates that the fairness mitigation algorithms meaningfully improve the fairness of the results.

Appendix XII
Train-test split sensitivity analysis – performance results

Figure 5 shows the performance predictive results obtained using an 80:20 train-test split. The results show an average decrease of 4% in sensitivity and 5% in PPV.

Appendix XIII
Bucket size sensitivity analysis

Figure 6 shows the fairness mitigation results of the bukcets algorithm for different bucket sizes and sensitive features.

Appendix XIV
Overfitting levels sensitivity analysis - prediction distribution with XGBoost

Figure 7 shows the prediction distribution with an overfit XGBoost model.

Appendix XV
Overfitting levels sensitivity analysis - prediction distribution with Logistic Regression

Figure 8 shows the prediction distribution with an overfit XGBoost model.

References

[1] D. Pessach and E. Shmueli, "A Review on Fairness in Machine Learning," *ACM Comput. Surv.*, vol. 55, no. 3, pp. 51:1–51:44, Feb. 2022. [Online]. Available: https://dl.acm.org/doi/10.1145/3494672

[2] M. Hardt, E. Price, E. Price, and N. Srebro, "Equality of Opportunity in Supervised Learning," in *Advances in Neural Information Processing Systems*, vol. 29. Curran Associates, Inc., 2016. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2016/hash/9d2682367c3935defcb1f9e247a97c0d-Abstract.html

[3] H. He, H. Daume III, and J. M. Eisner, "Learning to Search in Branch and Bound Algorithms," in *Advances in Neural Information Processing Systems*, vol. 27. Curran Associates, Inc., 2014. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2014/hash/757f843a169cc678064d9530d12a1881-Abstract.html

TABLE II: Logistic regression coefficients - MIMIC

Feature	**Coefficient (95% CI)**
Age group 60-75	-0.88 (-1.13 – -0.05)
Age group over 75	-0.81 (-1.14 – 0.00)
Age groups under 60	-0.78 (-1.06 – 0.00)
Race Black/African American	-1.02 (-1.71 – -0.46)
Race Other	-0.79 (-1.96 – -0.24)
Race Unknown	-0.41 (-1.12 – 0.07)
Race White	-0.79 (-1.44 – -0.26)
Age group 60-75 - race Other	0.03 (-0.04 – 0.28)
Age group 60-75 - race White	0.01 (-0.64 – 0.21)
Age group over 75 - race Other	0.00 (-0.12 – 0.24)
Age group over 75 - race White	0.00 (-0.64 – 0.19)
Age group under 60 - race Black/African American	-0.30 (-0.89 – 0.00)
Age group under 60 - race Other	-0.21 (-0.51 – 0.00)
Age group under 60 - race White	-0.25 (-0.92 – 0.00)
Age group – race Other	0.00 (-0.47 – 0.07)
Pain Location Incisional	-1.40 (-1.58 – -1.18)
GCS Motor Response Obeys Commands	-0.98 (-1.10 – -0.87)
Turn - Turns by Self	-0.49 (-0.60 – -0.37)
Braden Friction Shear - No Apparent Problem	-0.49 (-0.58 – -0.35)
GCS Verbal Response Oriented	-0.31 (-0.43 – -0.21)
Age	0.01 (0.00 – 0.02)
BUN – last	0.00 (0.00 – 0.02)
RDW min	0.18 (0.14 – 0.20)
Strength L Leg Full resistance	-0.27 (-0.39 – -0.16)

TABLE III: Logistic regression coefficients - GOSSIS

Feature	**Coefficient (95% CI)**
Age group 60-75	-0.20 (-0.36 – 0.00)
Age group over 75	0.02 (-0.10 – 0.35)
Age group under 60	0.00 (-0.04 – 0.20)
Apache 3j bodysystem Cardiovascular	-0.24 (-0.47 – -0.05)
Apache 3j bodysystem Gastrointestinal	0.00 (-0.19 – 0.07)
Apache 3j bodysystem Metabolic	-1.03 (-1.49 – -0.69)
Apache 3j bodysystem Neurological	0.07 (-0.07 – 0.19)
Apache 3j bodysystem Other/Unknown	0.20 (0.00 – 0.34)
Apache 3j bodysystem Respiratory	-0.23 (-0.46 – -0.10)
Apache 3j bodysystem Sepsis	0.18 (0.00 – 0.31)
Age group 60-75 – Apache 3j bodysystem Cardiovascular	0.09 (-0.01 – 0.29)
Age group 60-75 – Apache 3j bodysystem Respiratory	0.09 (-0.09 – 0.41)
Age group 60-75 – Apache 3j bodysystem Sepsis	0.51 (0.23 – 0.84)
Age group – Apache 3j bodysystem Other/Unknown	0.18 (0.00 – 0.38)
Age group over 75 – Apache 3j bodysystem Cardiovascular	-0.06 (-0.26 – 0.02)
Age group under 60 – Apache 3j bodysystem Cardiovascular	0.00 (-0.16 – 0.12)
Age group under 60 – Apache 3j bodysystem Metabolic	-0.32 (-0.85 – 0.04)
Age group under 60 – Apache 3j bodysystem Neurological	-0.13 (-0.42 – 0.03)
Age group under 60 – Apache 3j bodysystem Other/Unknown	0.00 (-0.27 – 0.30)
Age group under 60 – Apache 3j bodysystem Respiratory	-0.25 (-0.52 – 0.00)
d1 spo2 min	-0.03 (-0.03 – -0.03)
gcs motor Apache	-0.30 (-0.34 – -0.28)
d1 resprate max	0.02 (0.01 – 0.02)
d1 bun max	0.01 (0.00 – 0.02)
age	0.02 (0.01 – 0.03)
d1 bun min	0.00 (0.00 – 0.01)
Apache 3j diagnosis	0.00 (0.00 – 0.00)
urineoutput Apache	0.00 (0.00 – 0.00)
Ventilated Apache	1.12 (1.01 – 1.22)
gcs verbal Apache	-0.09 (-0.12 – -0.05)

TABLE IV: Post-processing runtime

		Iterative algorithm	Buckets algorithm	Branch and bound algorithm
GOSSIS	Intersection of age and APACHE body system	00:00:34	00:02:08	25:44:56
MIMIC	Intersection of age and race	00:00:22	00:01:37	00:01:43

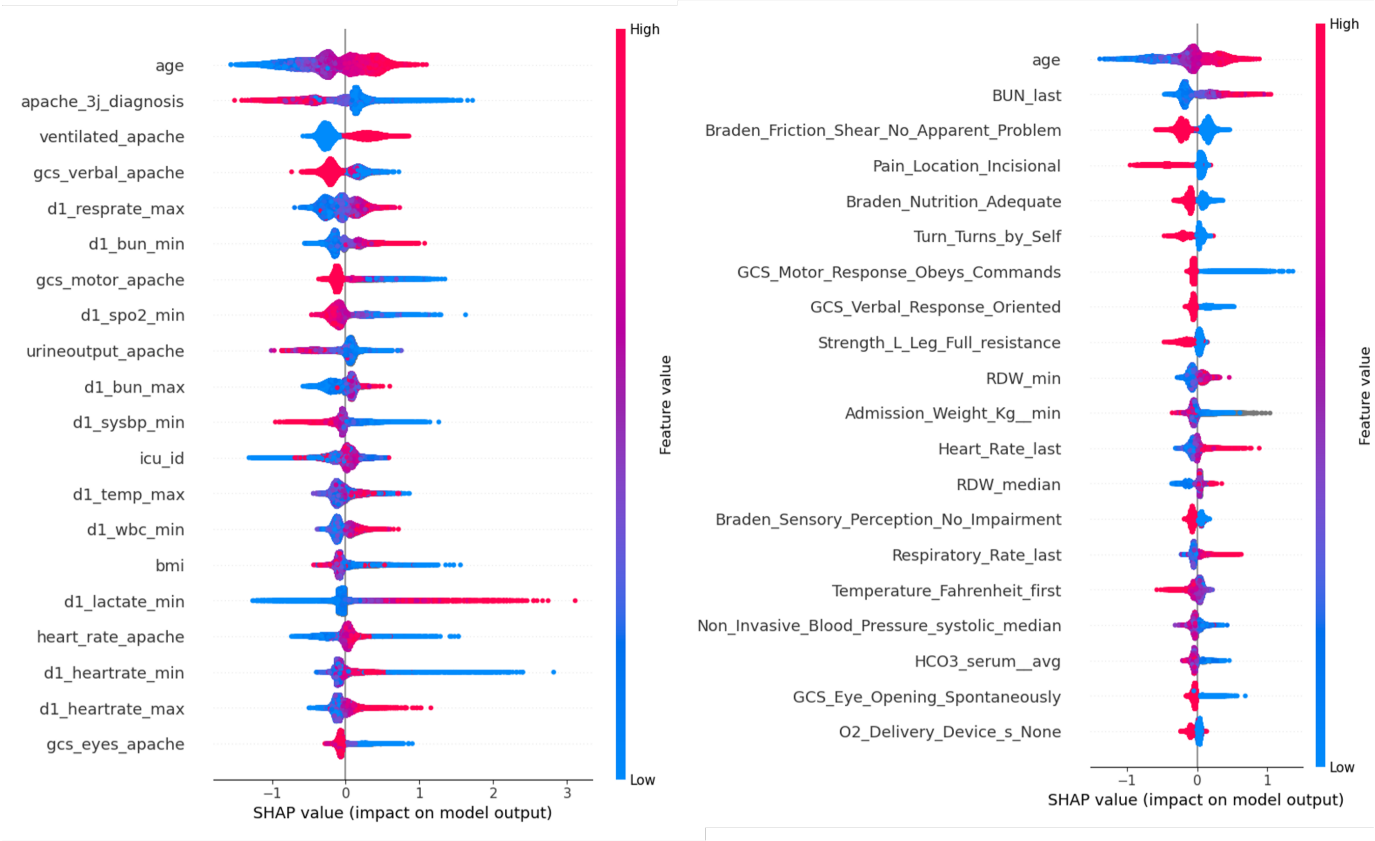

Fig. 1: SHAP feature importance for the GOSSIS (left) and MIMIC (right) data

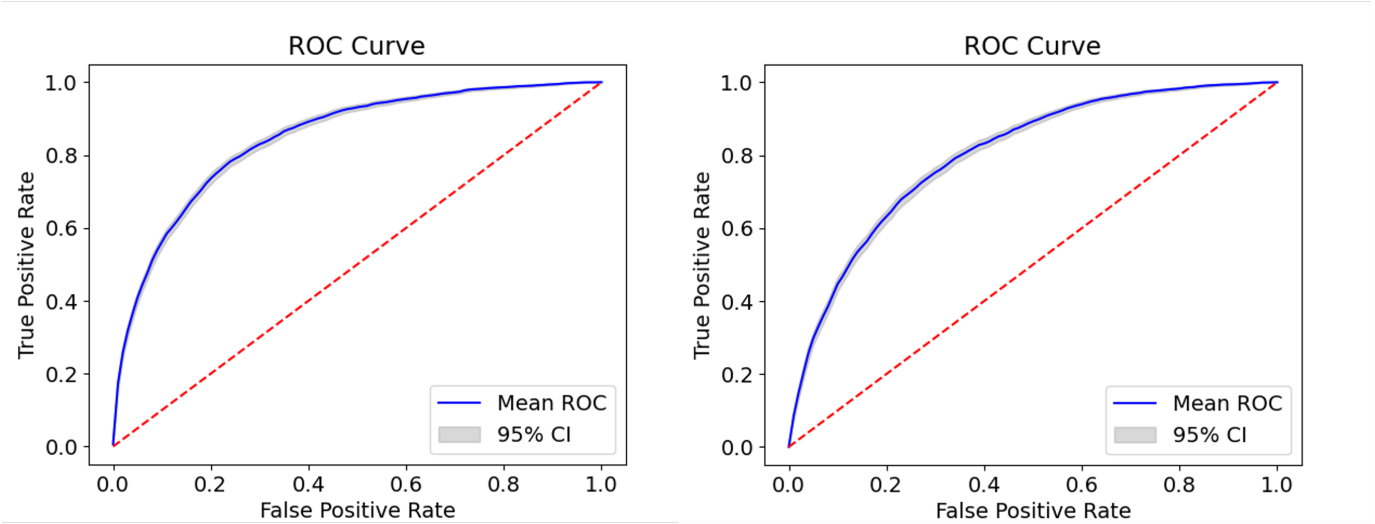

Fig. 2: ROC curve for a model trained on the GOSSIS data (left) and the MIMIC data (right)

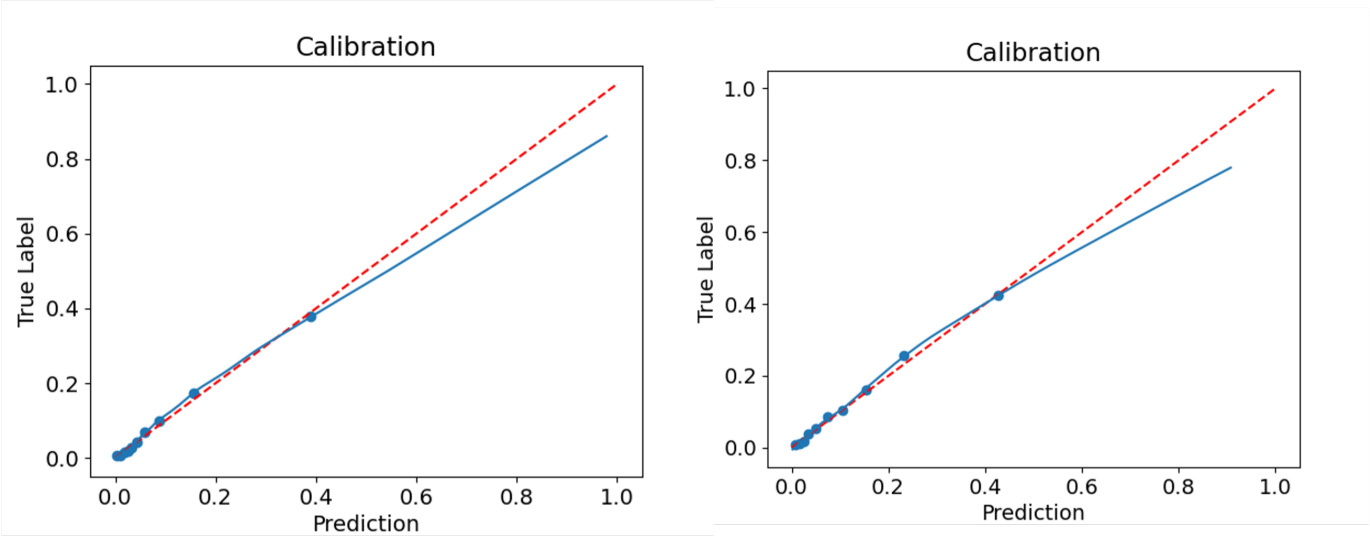

Fig. 3: Calibration curve for a model trained on the GOSSIS data (left) and the MIMIC data (right)

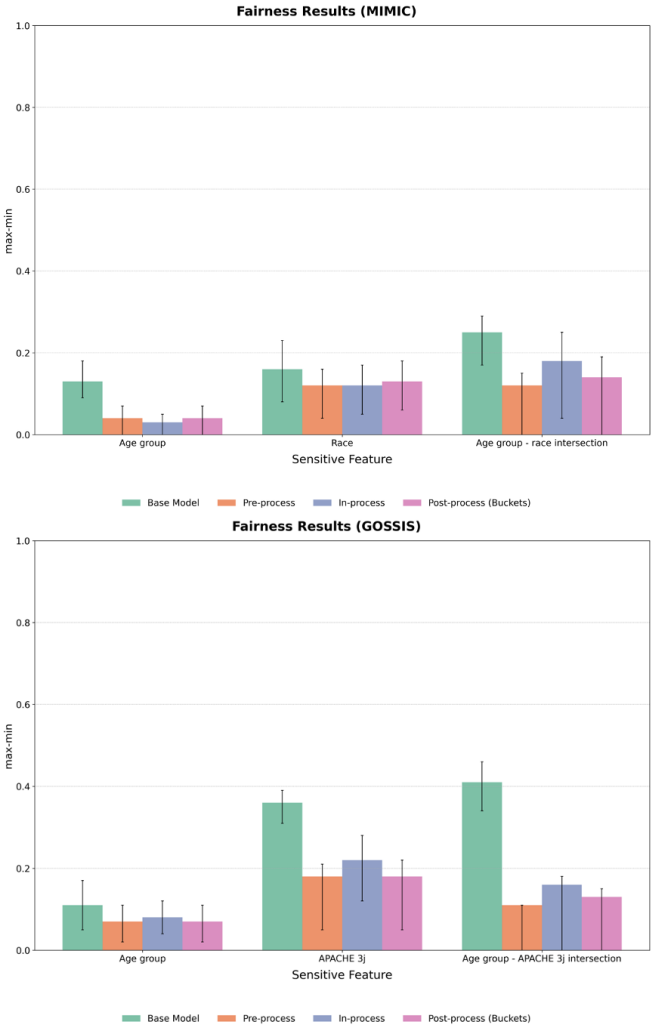

Fig. 4: Fairness results with an 80:20 train-test split (Logistic Regression)

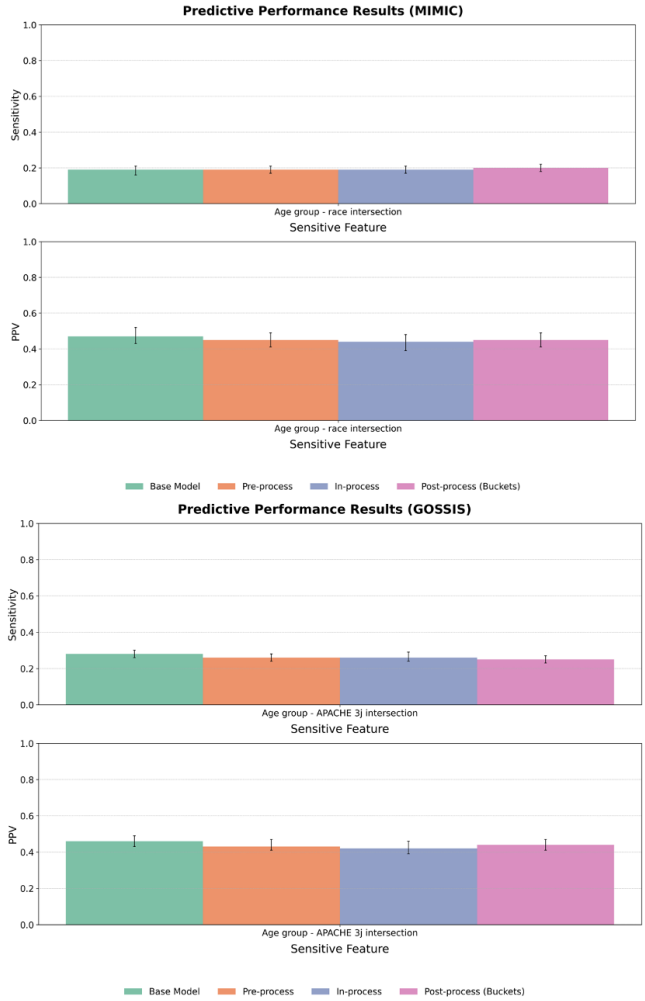

Fig. 5: Performance results with an 80:20 train-test split (Logistic Regression)

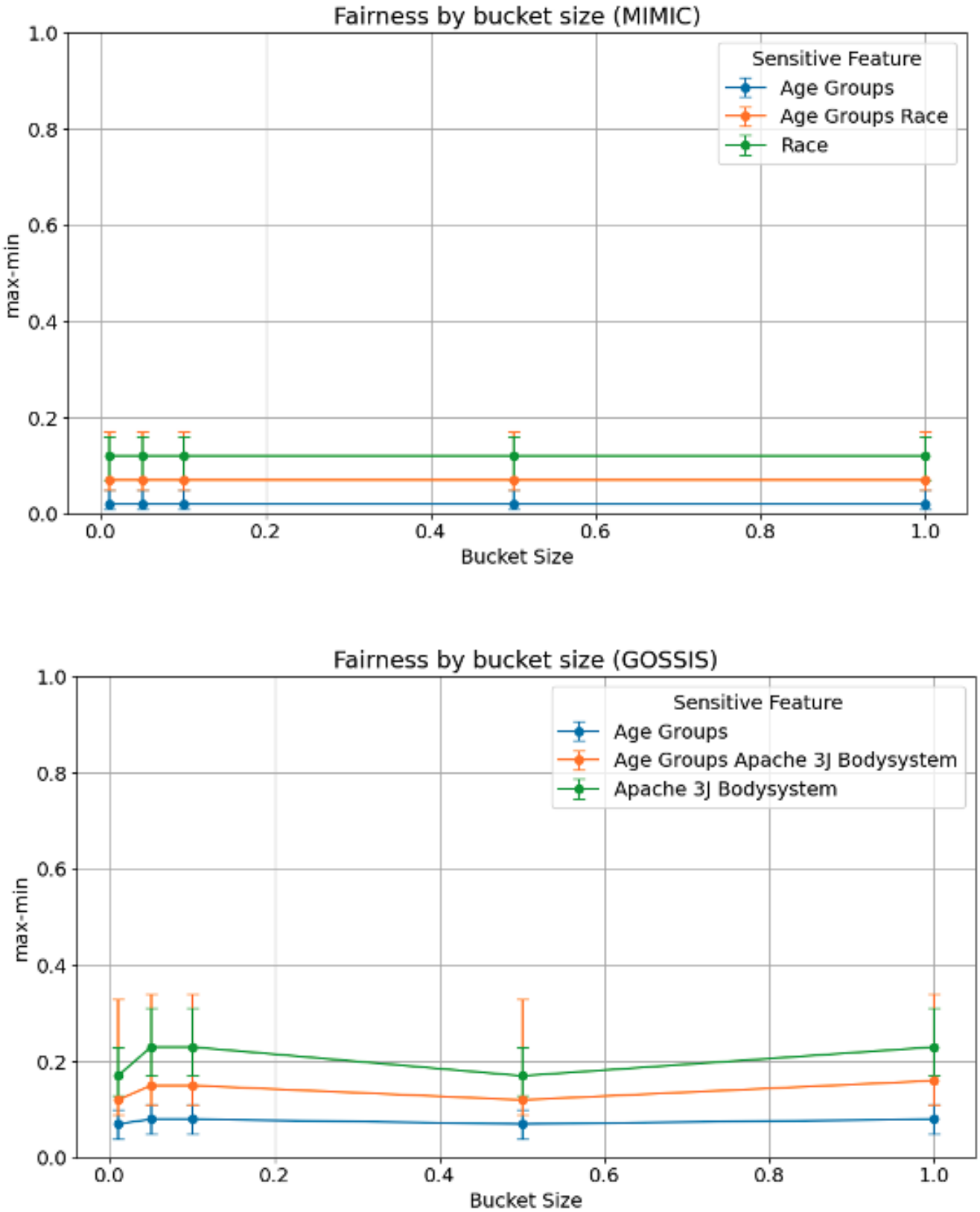

Fig. 6: Fairness results by bucket size (logistic regression)

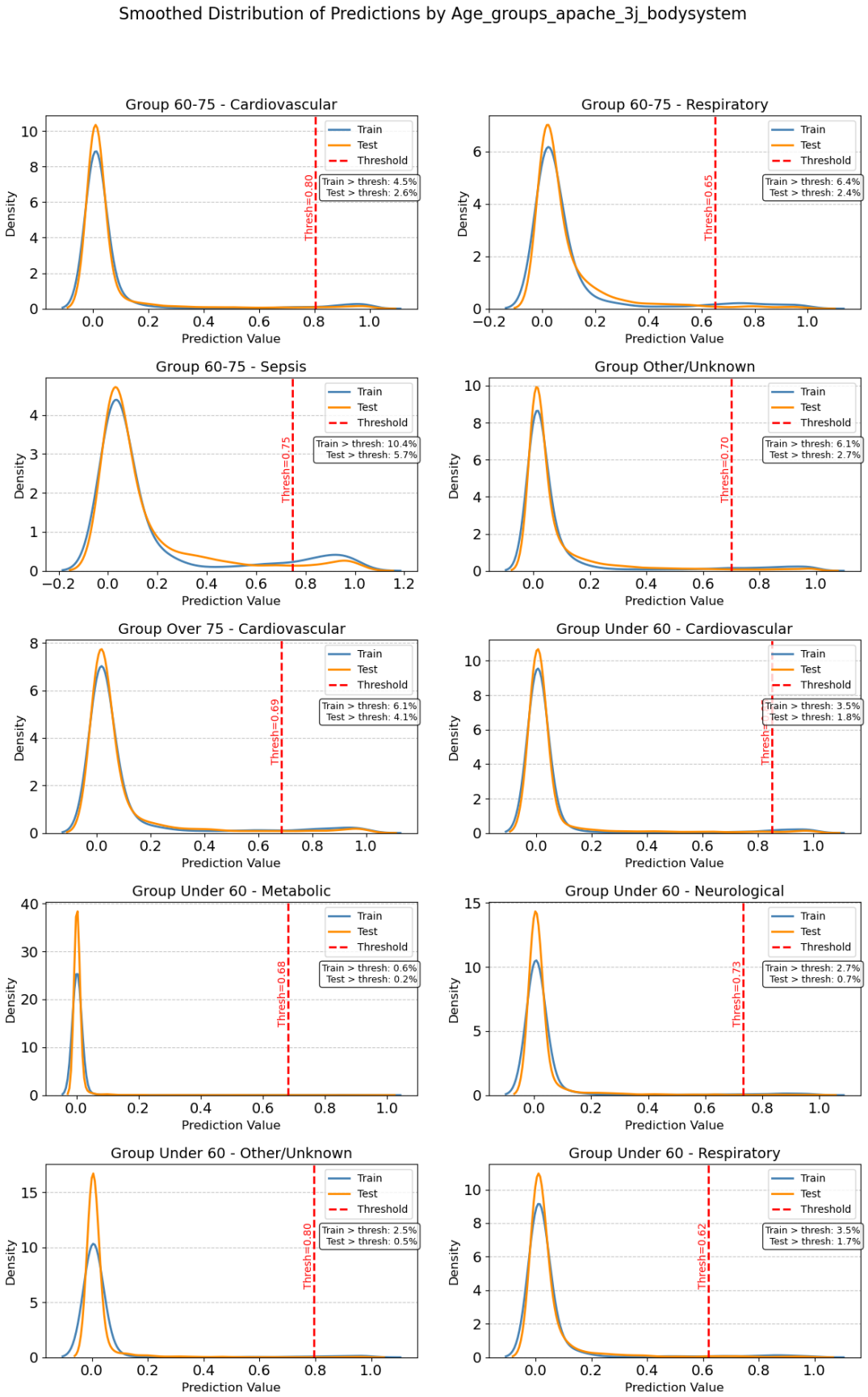

Fig. 7: XGBoost prediction distribution on the GOSSIS data

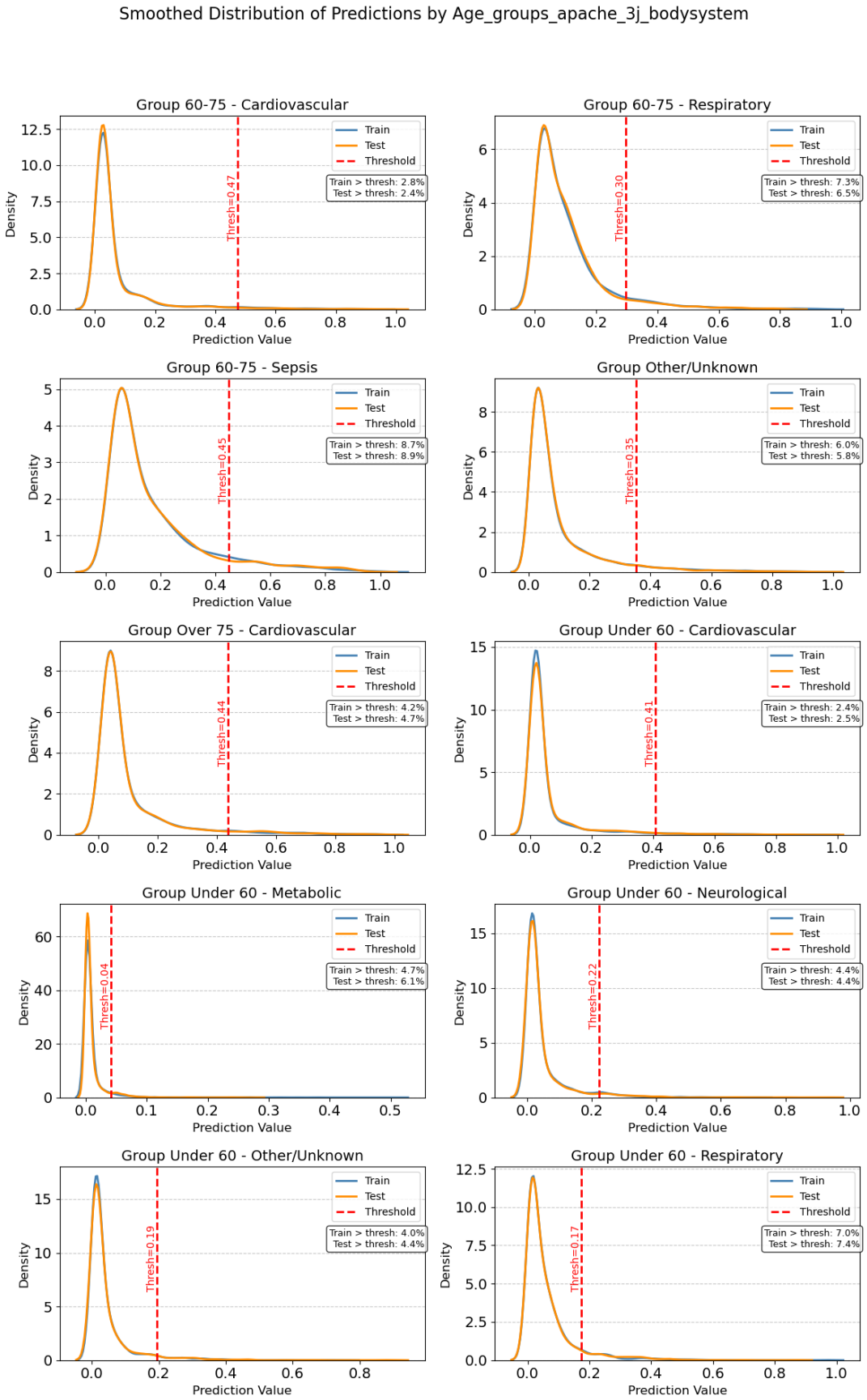

Fig. 8: Logistic Regression prediction distribution on the GOSSIS data